



Chitkara

PAPER OF CSE-2025 - PYTHON TEST7 EXAM

Q1. Astronaut Log Cleanup System

On a mission to Jupiter's moons, astronauts keep digital logs of everything they observe. A sudden radiation surge corrupts their log entries, mixing in digits, control symbols, and system noise. You are assigned to clean these logs so mission control can read them clearly.

For every corrupted log entry:

1. Keep only alphabetic characters (A–Z, a–z).
2. Convert all kept letters to lowercase.
3. If the cleaned entry has **4 or more letters**, reverse it.
4. If it has **fewer than 4 letters**, keep it unchanged.
5. Store all non-empty cleaned entries in a new list.

Your Task

Write a function **repair_logs(entries: list) -> list** that returns the cleaned astronaut log entries.

Input Format

A list of strings that may include letters, numbers, and symbols

Output Format

A list of cleaned and processed log entries

Sample Input 1

```
["Sp@ac3e", "Mi$$oon", "R0ck!et", "Io!"]
```

Sample Output 1

```
['ecaps', 'noosim', 'tekcr', 'io']
```

Sample Input 2

```
["L@aun9ch", "St@ar1", "O!rb2it", "Ga$s"]
```

Sample Output 2

```
['hcnuai', 'rats', 'tibro', 'gas']
```

ANSWER:

Test case 1 :

Input:

`["L@aun9ch", "St@ar1", "O!rb2it", "Ga$s"]`

Output:

`['hcnu!l', 'rats', 'tibro', 'gas']`Test case 2 :

Input:

`["A1", "b2c", "d3e4f", "Hi!!", "xyz"]`

Output:

`['a', 'bc', 'def', 'hi', 'xyz']`Test case 3 :

Input:

`["mithril", "Ar@gon", "f3rodO", "#elf"]`

Output:

`['lirhtim', 'nogra', 'odorf', 'elf']`Test case 4 :

Input:

`["$@#", "12345", "!!!"]`

Output:

`[]`Test case 5 :

Input:

`["p3alantir", "va!lin0or", "num3enor"]`

Output:

`['ritnalap', 'ronilav', 'ronemun']`Test case 6 :

Input:

`["a", "B", "c2", "D!e", "f3g!h"]`

Output:

`['a', 'b', 'c', 'de', 'fgh']`

Q2. Underwater Research Data Cleaner

Marine biologists exploring deep-sea trenches record organism names using submersible scanners. But due to high-pressure interference, the scanner outputs get mixed with salt-stains, digits, and sensor glitches. Your task is to recover the proper organism names from these corrupted recordings.

For each corrupted organism string:

1. Keep only alphabetic letters (A–Z, a–z).
2. Convert all letters to lowercase.
3. If the cleaned name has **4 or more letters**, reverse it.
4. If it has **fewer than 4 letters**, keep it unchanged.

5. Collect all non-empty cleaned names into a list.

Your Task

Write a function **clean_marine_data(records: list) -> list** that returns the cleaned list of organism names.

Input Format

A list of strings possibly containing letters, digits, and symbols

Output Format

A list of cleaned and processed organism names

Sample Input 1

```
["Sh@a1rk", "C0o#ra1", "!Eel", "Oct!0o"]
```

Sample Output 1

```
['krahs', 'laroc', 'eel', 'otco']
```

Sample Input 2

```
["T@u2na", "Se@a#l", "Kr!i3ll", "Sh0e!!!l"]
```

Sample Output 2

```
['anut', 'laes', 'llirk', 'llehs']
```

ANSWER:

Test case 1 :

Input:

```
["T@u2na", "Se@a#l", "Kr!i3ll", "Sh0e!!!l"]
```

Output:

```
['anut', 'laes', 'llirk', 'llehs']
```

Test case 2 :

Input:

```
["A1", "b2c", "d3e4f", "Hi!!", "xyz"]
```

Output:

```
['a', 'bc', 'def', 'hi', 'xyz']
```

Test case 3 :

Input:

```
["mithril", "Ar@gon", "f3rodO", "#elf"]
```

Output:

```
['lirhtim', 'nogra', 'odorf', 'elf']
```

Test case 4 :

Input:

```
["$@#", "12345", "!!!"]
```

Output:

```
[]
```

Test case 5 :

Input:

```
["p3alantir", "va!lin0or", "num3enor"]
```

Output:

```
['ritnalap', 'ronilav', 'ronemun']
```

Test case 6 :

Input:

```
["a", "B", "c2", "D!e", "f3g!h"]
```

Output:

```
['a', 'b', 'c', 'de', 'fgh']
```

Q3. Museum Artifact Label Restorer

In the National Museum of Lost Civilizations, artifact labels have decayed over centuries. Many of them now contain dust marks, random digits, and scratches that hide the original names. You are the Restoration Expert responsible for reconstructing these names.

For each damaged label:

1. Keep only alphabetic letters (A–Z, a–z).
2. Convert all letters to lowercase.
3. If the cleaned label has **4 or more letters**, reverse it.
4. If it has **fewer than 4 letters**, keep it unchanged.
5. Collect all non-empty cleaned labels into a list.

Your Task

Write a function **restore_labels(labels: list) -> list** that reconstructs and returns the cleaned artifact names.

Input Format

A list of strings with letters, digits, and/or symbols

Output Format

A list of cleaned and processed artifact names

Sample Input 1

```
["St@a1tue", "M0a!sk", "Ur$n", "T0o#mb"]
```

Sample Output 1

```
['eutats', 'ksam', 'urn', 'b moot']
```

Sample Input 2

```
["Cr0o!wn", "S!pe3ar", "G@e#m", "Jad2e"]
```

Sample Output 2

```
['nworc', 'raeps', 'gem', 'edaj']
```

ANSWER:

Test case 1 :

Input:

```
["Cr0o!wn", "S!pe3ar", "G@e#m", "Jad2e"]
```

Output:

```
['nworc', 'raeps', 'gem', 'edaj']
```

Test case 2 :

Input:

```
["A1", "b2c", "d3e4f", "Hi!!", "xyz"]
```

Output:

```
['a', 'bc', 'def', 'hi', 'xyz']
```

Test case 3 :

Input:

```
["mithril", "Ar@gon", "f3rodO", "#elf"]
```

Output:

```
['lirhtim', 'nogra', 'odorf', 'elf']
```

Test case 4 :

Input:

```
["$@#", "12345", "!!!"]
```

Output:

```
[]
```

Test case 5 :

Input:

```
["p3alantir", "va!lin0or", "num3enor"]
```

Output:

```
['ritnalap', 'ronilav', 'ronemun']
```

Test case 6 :

Input:

```
["a", "B", "c2", "D!e", "f3g!h"]
```

Output:

```
['a', 'b', 'c', 'de', 'fgh']
```

Q4. Pirate Map Cipher Decoder

On a **deserted island**, explorers uncover an old pirate chest containing scraps of a torn treasure map. Some scraps contain real location names, while others have faded numbers, weathered symbols, or ink smudges. A hidden note inside the chest reveals the rule pirates used to hide their treasure route in coded form.

Your Task

To decode the pirate's treasure route, follow these rules in the exact order the scraps appear:

1. Read each scrap in sequence.
2. A valid location scrap contains only alphabetic letters (A–Z, a–z).
3. If its length is odd, extract its first letter.
4. If its length is even, extract its last letter.
5. Keep the extracted letters in the same order.
6. Convert the final string to UPPERCASE.

Input Format:

A list of strings collected from the map scraps.
Only alphabetic scraps count as valid location names.

Output Format:

A single uppercase string representing the decoded route.

Sample Input 1:

```
["cove", "reef", "99", "@", "lagoon", "shore"]
```

Sample Output 1:

```
EFNS
```

Processing:

- cove → even → e
- reef → even → f
- lagoon → even → n
- shore → odd → s

```
→ "efns" → "EFNS"
```

Sample Input 2:

```
["$$", "54" "!!", "****"]
```

Sample Output 2:

```
""
```

Explanation: None of element is alphabet therefore, Return Empty String

ANSWER:

Test case 1 :

Input:

["cove", "reef", "99", "@", "lagoon", "shore"]

Output:

EFNS

Test case 2 :

Input:

["red", "blue", "123", "star", "moon", "!"]

Output:

RERN

Test case 3 :

Input:

["sun", "earth", "pluto", "78", "galaxy"]

Output:

SEPY

Test case 4 :

Input:

["ion", "@", "proton", "electron", "muon"]

Output:

INNN

Test case 5 :

Input:

["rock", "lava", "gas", "plasma", "123x", "ion"]

Output:

KAGAI

Test case 6 :

Input:

["zen", "yogi", "mono", "plane", "rt", "\$\$"]

Output:

ZIOPT

Q5. Time Travellers Chrono-Note Decoder

A time traveller returns with a damaged **chrono-notebook** containing mixed entries from different eras. Some entries are complete historical terms, while others are glitched timestamps, corrupt futuristic symbols, or numbers distorted during time jumps. A note on the device describes a rule for reconstructing the “Chrono Key”, a hidden message across timelines.

Your Task

To decode the Chrono Key, apply the rules **in the exact sequence the entries appear**:

1. Read each entry from the list in order.
2. Treat only alphabetic strings (A–Z, a–z) as valid historical terms.
3. If the term has an **odd** number of characters → extract its **first letter**.
4. If it has an **even** number of characters → extract its **last letter**.
5. Append all extracted letters in the same order.
6. Convert the final result to **UPPERCASE**.

Input Format:

A list of mixed entries recorded across timelines.
Only alphabetic entries count as valid.

Output Format:

A single uppercase string – the decoded Chrono Key.

Sample Input 1:

```
["roman", "404", "empire", "age", "!", "future"]
```

Sample Output 1:

```
REAE
```

Processing:

- roman → odd → r
- empire → even → e
- age → odd → a
- future → even → e

Collected → "reae" → **"REAE"**

Sample Input 2:

```
["###", "84" "^^", "****"]
```

Sample Output 2:

```
""
```

Explanation: None of element is alphabet therefore, Return Empty String

ANSWER:

Test case 1 :

Input:

```
["roman", "404", "empire", "age", "!", "future"]
```

Output:

```
REAE
```


Test case 2 :

Input:

["red", "blue", "123", "star", "moon", "!"]

Output:

RERN

Test case 3 :

Input:

["sun", "earth", "pluto", "78", "galaxy"]

Output:

SEPY

Test case 4 :

Input:

["ion", "@", "proton", "electron", "muon"]

Output:

INNN

Test case 5 :

Input:

["rock", "lava", "gas", "plasma", "123x", "ion"]

Output:

KAGAI

Test case 6 :

Input:

["zen", "yogi", "mono", "plane", "rt", "\$\$"]

Output:

ZIOPT

Q6. Magic Academy Spellbook Decoder

At the **Arcane Academy**, apprentices discover an old spellbook filled with mixed entries. Some lines are real spell names, while others are page numbers, scribbled notes, or strange arcane symbols. A note on the first page explains a secret method to extract a hidden master spell from the spellbook entries.

Your Task

To reveal the hidden master spell, follow these rules **in the exact order the entries appear**:

1. Read each entry from the list one by one.
2. Treat an entry as a **valid spell** only if it contains **letters only** (A–Z, a–z).
3. If the spell's length is **odd**, take its **first letter**.
4. If the spell's length is **even**, take its **last letter**.
5. Collect the letters in the same order and convert the final string to **UPPERCASE**.

Function Signature:

```
def decode_spellbook_words(entries):
```

Input Format:

A list of strings representing spellbook entries.
Only entries with alphabetic characters only are treated as valid spells.

Output Format:

A single uppercase string representing the decoded master spell.

Sample Input 1:

```
["spark", "42", "flame", "orb", "!", "rune"]
```

Sample Output 1:

```
SFOE
```

Processing:

- spark → 5 (odd) → take **s**
- flame → 5 (odd) → take **f**
- orb → 3 (odd) → take **o**
- rune → 4 (even) → take **e**

Collected: "sfoe" → **"SFOE"**

Sample Input 2:

```
["987", "$$" " !8", "****"]
```

Sample Output 2:

```
""
```

Explanation: None of element is alphabet therefore, Return Empty String

ANSWER:

Test case 1 :

Input:

```
["spark", "42", "flame", "orb", "!", "rune"]
```

Output:

```
SFOE
```

Test case 2 :

Input:

```
["red", "blue", "123", "star", "moon", " !"]
```

Output:

```
RERN
```

Test case 3 :

Input:

["sun", "earth", "pluto", "78", "galaxy"]

Output:

SEPY

Test case 4 :

Input:

["ion", "@", "proton", "electron", "muon"]

Output:

INNN

Test case 5 :

Input:

["rock", "lava", "gas", "plasma", "123x", "ion"]

Output:

KAGAI

Test case 6 :

Input:

["zen", "yogi", "mono", "plane", "rt", "\$\$"]

Output:

ZIOPT

Q7. Dholakpur Royal School Marks Evaluator

After a storm struck the palace archives, several student records at the Royal School in Dholakpur were corrupted — some names turned into numbers, some marks became words, and some entries got mixed up. Princess Indumati and Chhota Bheem need your help to calculate the average marks of all valid students.

Your Task

Write a function:

```
def dholakpur_average_marks(records: dict) -> str
```

The function must:

- Consider only **valid records** (key = string, value = integer)
- Skip all invalid entries
- Use exception handling:

Exception Rules

- If the input is **not a dictionary**, raise `TypeError` and return "Invalid Data"
- If **no valid records** exist, raise `ZeroDivisionError` and return "No Valid Records"

Return the final result in the format:

Average Marks: X.XX

(rounded to 2 decimals)

Input Format

A dictionary containing student names as keys and marks as values.
Some entries may be corrupted.

Output Format

One of the following:

- Average Marks: X.XX
- Invalid Data
- No Valid Records

Sample Input 1

```
{"Bheem": 95, "Chutki": 90, "Raju": "brave", "Jaggu": 88}
```

Sample Output 1

Average Marks: 91.00

Sample Input 2

12345

Sample Output 2

Invalid Data

ANSWER:

Test case 1 :

Input:

```
{"Bheem": 95, "Chutki": 90, "Raju": "brave", "Jaggu": 88}
```

Output:

Average Marks: 91.00

Test case 2 :

Input:

```
{"Snape": 90, "Voldemort": 100, 123: 85}
```

Output:

Average Marks: 95.00

Test case 3 :

Input:

```
{}
```

Output:

No Valid Records

Test case 4 :

Input:

```
{"Sirius": "A+", "Remus": 85, "Peter": 75}
```

Output:

Average Marks: 80.00

Test case 5 :

Input:

"Invalid text"

Output:

Invalid Data

Test case 6 :

Input:

{"Ginny": 92, "Fred": 88, "George": None, "Percy": 91, "Charlie": "N/A"}

Output:

Average Marks: 90.33

Q8. Pokemon Trainer Academy Marks Calculator

After Team Rocket attempted to hack the system, several trainee records at the Pokémon Trainer Academy became corrupted — some trainer names turned into numbers, some marks turned into strings, and some fields glitched entirely. Professor Oak needs your help to restore the records by calculating the average marks of all valid trainers.

Your Task

Write a function:

```
def trainer_average_marks(records: dict) -> str
```

The function must:

- Consider only **valid records** (key = string, value = integer)
- Skip all invalid entries
- Use exception handling:

Exception Rules

- If the input is **not a dictionary**, raise `TypeError` and return "Invalid Data"
- If **no valid records** exist, raise `ZeroDivisionError` and return "No Valid Records"

Return the final result in the format:

Average Marks: X.XX

(rounded to 2 decimals)

Input Format

A dictionary containing trainer names as keys and marks as values.
Some entries may be corrupted.

Output Format

One of the following:

- Average Marks: X.XX
- Invalid Data
- No Valid Records

Sample Input 1

```
{"Ash": 95, "Misty": 88, "Brock": "solid", "Serena": 90}
```

Sample Output 1

Average Marks: 91.00

Sample Input 2

12345

Sample Output 2

Invalid Data

ANSWER:

Test case 1 :

Input:

```
{"Ash": 95, "Misty": 88, "Brock": "solid", "Serena": 90}
```

Output:

Average Marks: 91.00

Test case 2 :

Input:

```
{"Snape": 90, "Voldemort": 100, 123: 85}
```

Output:

Average Marks: 95.00

Test case 3 :

Input:

```
{}
```

Output:

No Valid Records

Test case 4 :

Input:

```
{"Sirius": "A+", "Remus": 85, "Peter": 75}
```

Output:

Average Marks: 80.00

Test case 5 :

Input:

"Invalid text"

Output:

Invalid Data

Test case 6 :

Input:

`{"Ginny": 92, "Fred": 88, "George": None, "Percy": 91, "Charlie": "N/A"}`

Output:

Average Marks: 90.33

Q9. Sherlock Holmes Detective Training Evaluator

After Professor Moriarty hacked the academy's system, several trainee records at the Baker Street Detective Academy became corrupted — some trainee names turned into numbers, and some marks became cryptic strings. Sherlock Holmes needs your help to restore order by calculating the average valid marks.

Your Task

Write a function:

```
def detective_average_marks(records: dict) -> str
```

The function must:

- Consider only **valid records**
- (key = string, value = integer)
- Skip all invalid entries
- Use exception handling:

Exception Rules

- If the input is **not a dictionary**, raise `TypeError` and return **"Invalid Data"**
- If **no valid records** exist, raise `ZeroDivisionError` and return **"No Valid Records"**

Return the final result in the format:

Average Marks: X.XX

(rounded to 2 decimals)

Input Format

A dictionary containing trainee names and their marks.
Some entries may be corrupted.

Output Format

One of the following:

- Average Marks: X.XX
- Invalid Data
- No Valid Records

Sample Input 1`{"Sherlock": 98, "Watson": 92, "Moriarty": "chaos", "Hudson": 85}`

Sample Output 1

Average Marks: 91.67

Sample Input 2

12345

Sample Output 2

Invalid Data

ANSWER:Test case 1 :

Input:

{"Sherlock": 98, "Watson": 92, "Moriarty": "chaos", "Hudson": 85}

Output:

Average Marks: 91.67

Test case 2 :

Input:

{"Snape": 90, "Voldemort": 100, 123: 85}

Output:

Average Marks: 95.00

Test case 3 :

Input:

{}

Output:

No Valid Records

Test case 4 :

Input:

{"Sirius": "A+", "Remus": 85, "Peter": 75}

Output:

Average Marks: 80.00

Test case 5 :

Input:

"Invalid text"

Output:

Invalid Data

Test case 6 :

Input:

{"Ginny": 92, "Fred": 88, "George": None, "Percy": 91, "Charlie": "N/A"}

Output:

Average Marks: 90.33